

*DRAFT: IntelliJ + Maven: API Test Automation – Local Setup



Scope: Local developer setup for the **API test automation** project (JsonAPI-based framework). Covers prerequisites, cloning/import, IntelliJ config, POM/profile setup, running from CLI/IntelliJ, overriding properties, logging, parallelism, debugging, data/SQL usage, and FAQs.

0) Overview (how this framework works)

- **Tech:** Java 17, Maven, TestNG, RestAssured, JSON POJOs.
- **Structure:** A reusable **framework** (parent + resources) that each **project** depends on.
- **Config** is property-file driven and loaded at runtime (environment host optional extra). A **ResourceManager** listener wires configs/branding/SQL at suite start.

1) Prerequisites

- **Java 17+ JDK** installed; set JAVA_HOME. Verify with `java -version`.
- **Maven 3.9+** (`mvn -v`).
- **Git** and **IntelliJ IDEA** (Community or Ultimate).
- **Nexus** read access and team `~/.m2/settings.xml` (server creds + repos) placed at:
 - Windows: `C:\Users\<you>\.m2\settings.xml`
 - macOS/Linux: `/Users/<you>/.m2/settings.xml` or `/home/<you>/.m2/settings.xml`

2) Clone the test project repo

Pick one method.

GitLab Repo Location: <https://gitlab.com/ascensus-gs/products/depot/qa-automation/api-test-automation>

A) IntelliJ – Get from Version Control

1. Open IntelliJ **Get from VCS**.
2. Paste the GitLab URL choose a workspace folder **Clone**.
3. Authenticate (PAT/SSO) when prompted.

B) Git CLI

```
cd <your-workspace>
git clone <gitlab repo url>
cd <repo>
```

C) SourceTree

1. **Clone** paste the repo URL pick destination **Clone**.

3) Import & IDE configuration (Maven project)

1. IntelliJ: **File Open** select repo root **Open as Project**.
2. When prompted, allow Maven import; download sources/docs.
3. **Project SDK:** *File Project Structure Project SDK: Java 17; Language level: 17.*
4. **Maven JDK:** *Settings Build Tools Maven* set **JDK 17** for Maven runner.
5. Recommended plugins: **TestNG**, **Lombok** (if used), **.ignore**.

*Tip: If classes are red, click **Maven Reimport**. If stale indexes: **File Invalidate Caches / Restart**.*

4) Project POM essentials

Your **test project** should:

- **Extend the JsonAPI parent:**

```

<parent>
  <groupId>jsonapi</groupId>
  <artifactId>jsonapi-parent</artifactId>
  <version>1.0.0</version>
</parent>

```

- **Depend on framework** (code + resources):

```

<dependency>
  <groupId>jsonapi</groupId>
  <artifactId>jsonapi</artifactId>
  <version>1.0.0</version>
  <type>jar</type>
</dependency>
<dependency>
  <groupId>jsonapi</groupId>
  <artifactId>jsonapi</artifactId>
  <version>1.0.0</version>
  <type>jar</type>
  <classifier>resources</classifier>
  <scope>provided</scope>
</dependency>

```

- **Declare host properties default:**

```

<properties>
  <host.properties>${HOSTNAME}.properties</host.properties>
</properties>

```

- **Surefire plugin** passes system props + suite XML:

```

<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-surefire-plugin</artifactId>
  <version>3.2.5</version>
  <configuration>
    <systemPropertyVariables>
      <hostProperties>${host.properties}</hostProperties>
      <envProperties>${environment.properties}</envProperties>
    </systemPropertyVariables>
    <suiteXmlFiles>
      <suiteXmlFile>${xml.suite}</suiteXmlFile>
    </suiteXmlFiles>
  </configuration>
  <dependencies>
    <dependency>
      <groupId>org.apache.maven.surefire</groupId>
      <artifactId>surefire-testng</artifactId>
      <version>3.2.5</version>
    </dependency>
  </dependencies>
</plugin>

```

- **Unpack framework resources/samples/docs** on demand:

```

<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-dependency-plugin</artifactId>
  <executions>
    <execution>
      <id>unpack-resources</id>
      <phase>generate-resources</phase>
      <goals><goal>unpack</goal></goals>
      <configuration>
        <artifactItems>
          <artifactItem>
            <groupId>jsonapi</groupId>
            <artifactId>jsonapi</artifactId>
            <version>1.0.0</version>
            <classifier>resources</classifier>
            <type>jar</type>
            <outputDirectory>${basedir}/src/test/resources</outputDirectory>
          </artifactItem>
        </artifactItems>
      </configuration>
    </execution>
    <execution>
      <id>unpack-samples</id>
      <phase>generate-resources</phase>
      <goals><goal>unpack</goal></goals>
      <configuration>
        <artifactItems>
          <artifactItem>
            <groupId>jsonapi</groupId>
            <artifactId>jsonapi</artifactId>
            <version>1.0.0</version>
            <classifier>samples</classifier>
            <type>jar</type>
            <outputDirectory>${basedir}/samples</outputDirectory>
          </artifactItem>
        </artifactItems>
      </configuration>
    </execution>
    <execution>
      <id>unpack-documentation</id>
      <phase>generate-resources</phase>
      <goals><goal>unpack</goal></goals>
      <configuration>
        <artifactItems>
          <artifactItem>
            <groupId>jsonapi</groupId>
            <artifactId>jsonapi</artifactId>
            <version>1.0.0</version>
            <classifier>documentation</classifier>
            <type>jar</type>
            <outputDirectory>${basedir}/docs</outputDirectory>
          </artifactItem>
        </artifactItems>
      </configuration>
    </execution>
  </executions>
</plugin>

```

- **Optional clean-up** (remove unpacked files on `mvn clean`): `maven-clean-plugin` with filesets for `/src/test/resources`, `/samples`, `/docs`.

5) Configuration files & load order

Place config in `src/test/resources/config/`:

- **Environment file** (e.g., `stage.properties`), loaded first via `envProperties`.
- **Host file** (default `${HOSTNAME}.properties`), loaded second via `hostProperties`.
- **Extra file** (optional), loaded last via `extra.properties`.

Override at runtime: `-DenvProperties=<file> -DhostProperties=<file> -Dextra.properties=<file>`

If the host file fails to load, framework falls back to `config.properties`.

6) TestNG suites & profiles

- **Default profile** (required): `id build`, `activeByDefault=true`, sets `skipTests=true` so modules build cleanly with `mvn install`.
- **Execution profiles** (add as needed): each sets

- `xml.suite` = comma-separated suite list (relative paths)
- `environment.properties` = env file name
- Optional: `host.properties`, `extra.properties`

Examples

```
# Build framework & project (no tests)
mvn clean install

# Acceptance profile (runs suite[s] defined in the profile)
mvn clean install -Paccept

# Force re-resolve deps from Nexus
mvn clean install -U

# Override suites on demand
mvn clean install -Dxml.suite=bin/smoke.xml

# Override property files on demand
mvn clean install -DenvProperties=stage.properties -DhostProperties=MYPC.properties
```

Running specific classes/methods

- In your suite XML, list classes normally. To run **specific methods**, add a `<methods>` block with `<include name="..." />`.

Parallelism

- Control with `parallel="tests|classes|methods"` and `thread-count="N"` in the suite XML.

7) Run from IntelliJ

Option A – Maven run config

- Create a **Maven** run configuration:
 - **Working dir**: project root
 - **Command**: `clean install`
 - **Profiles**: e.g., `accept`
 - **Env/Host overrides** (optional): add to **Arguments** as `-D...`

Option B – TestNG suite

- Rightclick your suite XML (e.g., `bin/accept.xml`) **Run**. Ensure surefire picks up the same `systemPropertyVariables` as in POM when running via Maven.

Debugging

- Add `-Dmaven.surefire.debug` to Maven **Runner VM Options** and attach IntelliJ debugger (port shown in console).
- Or run TestNG directly in **Debug** from the suite XML.

Logging

- Console level: `-Dlogging.console=DEBUG`
- File level (logs/execution.log): `-Dlogging.file=DEBUG`

8) Base test scaffolding you'll use

- ****Extend **BaseRequestTest** for setup hooks and helpers:
 - `setUpEnvironmentBeforeAll()` picks the env from properties.
 - `setUpBeforeAll(String branding)` sets plan **branding** and preloads plan attributes (ids, uuids, funds list).
 - `setUpDefaultBeforeEach()` resets the data generator before each test.
 - Common helpers: `setBranding()`, `getProperty()`, `getCodeProperty()`, `loadJsonFile()`, `assertThat(...)`, DB helpers (`getDatabaseConnection()`, `loadSqlFile()`, `executeQuery(...)`).
- **HTTP client**: Extend `HttpRestApiClient` per project to handle auth/headers and RestAssured logging/proxy toggles.

9) Test data, JSON templates & SQL

- **JSON templates** in `src/test/resources/json`; each template maps to a **POJO** (fields must match). Loading via tests autoreplaces **datagen** tags.
- **POJOs** extend `BasePOJO`; support annotations like `@ValidationKey`, `@ValidationIgnore`, `@ValidationMatchEmpty`, `@ValidationNotNull`, `@JsonProperty`, `@JsonIgnore`, `@UUID`, `@NameFormat`, `@DatabaseFormat`.
- **SQL files** in `src/test/resources/sql`; queries keyed as:
 - Line 1: `<query_key> =`
 - Line 2..N: query

- Blank line separator
- Map SELECT results directly to POJOs; for complex shapes, split queries or use **PojoMerger** to build composites.

10) Samples & docs (optional)

Copy framework **samples** and **documentation** into your project for quick starts:

```
mvn clean generate-resources
# copies resources src/test/resources, samples /samples, docs /docs
```

*Do **not** commit copied samples/docs to source control.*

11) Common errors & fixes (FAQ)

- **Surefire: ****testSuiteXmlFiles0 has null value**
Cause: xml.suite not set.
Fix: define a **default** or **execution** profile; or run with `-Dxml.suite=...`
- **Cannot instantiate class core.listener.ResourceManager**
Cause: config files failed to load.
Fix: verify POM systemPropertyVariables and that env/host files exist; quick-fix with `-DenvProperties=... -DhostProperties=...`
- **Validation list order differences**
Fix: add `@ValidationKey` to key fields in list POJOs.
- **DB field names don't match POJO fields**
Fix: use `@NameFormat` on POJO fields.
- **Date/UUID formatting from DB**
Fix: use `@DatabaseFormat(date = true|uuid = true|split = ",")` on target fields.
- **Date format expectations (UTC)**
Fix: call `setDefaultDateFormat(true)` in setup when you need UTC.

12) Quick reference

```
# Build project (skip tests via default profile)
mvn clean install

# Run acceptance profile (as defined in POM)
mvn clean install -Paccept

# Override suite (smoke only)
mvn clean install -Dxml.suite=bin/smoke.xml

# Override property files
mvn clean install -DenvProperties=stage.properties -DhostProperties=MYPC.properties

# Force dependency updates from Nexus
mvn clean install -U

# Debug forked tests
mvn -Dmaven.surefire.debug clean test
```

13) SDET Starter Syllabus (1 week)

Day 1–2: Tooling setup, import project, verify env/host property load, run `mvn clean install` and confirm suite discovery.

Day 3–4: Add a new TestNG profile + suite; load JSON template POJO; hit a nonprod API with `HttpRestApiClient`; validate with `POJOtoPOJO assertThat`.

Day 5: Add a SQL validation using POJO mapping; exercise `@ValidationKey` for list matching; try `-Dlogging.console=DEBUG` and parallel runs.

Appendix – File locations (typical)

- **Suites:** `/bin/*.xml`
- **Configs:** `/src/test/resources/config/*.properties`
- **JSON templates:** `/src/test/resources/json/*.json`
- **SQL:** `/src/test/resources/sql/*.sql`
- **Docs/Samples** (optional): `/docs`, `/samples`